Document number: P0237R1
Date: 2016-05-30
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: LEWG, SG14, SG6
Reply to: Vincent Reverdy <vince.rev@gmail.com>
Robert J. Brunner

Wording for fundamental bit manipulation utilities

Table of Contents

History and feedback

The idea of bit manipulation utilities has been originally proposed in the 2016 pre-Jacksonville mailing and has been discussed in both SG6 and LEWG in Jacksonville. The original document, P0237R0 fully describes and discusses the idea of fundamental bit manipulation utilities for the C++ language. Another proposal, P0161R0 had the same motivations but was focusing on bitsets. As P0237R0 seemed more generic and not restrained to the scope of `std::bitset`, we decided to push forward P0237R0, while still keeping in mind the functionalities and performances allowed by P0161R0. The genericity and abstraction offered by P0237R0 should not introduce a performance overhead (at least with the `-O2` optimization level on most compilers) when compared to P0161R0 and N3864.

The feedback from the Jacksonville meeting was positive: in top of their intrinsics functionalities, bit manipulation utilities have the potential to serve as a common basis for a `std::dynamic_bitset` replacement of `std::vector<bool>` and for bounded and unbounded precision arithmetic as discussed in N4038.

In terms of design, the following guidance was given by LEWG:

- Restrain `std::bit_reference` to unsigned fundamental integer types? [SF: 6, F: 3, N: 0, A: 0, SA :0]
- `std::bit_reference` provides no arithmetic except what the paper provides explicitly? [Unanimous consent]
- In small discussion groups, introducing `std::bit_value` was considered to be good idea

In Jacksonville, the following questions were raised:

- How to deal with word endianness?
- How to provide the possibility to get a mask from the bit position?
- How to deal with infinite ranges of bits, particularly for unbounded precision arithmetic?
- What bitwise logic operators should bits provide?
- How to make `std::bit_iterator` compatible with proxy iterators presented in P0022R1?

If bit utilities make it to the standard, they will be used, at least, as the basis for an implementation of a dynamic bitset, and are likely to be used as an interface to the underlying containers of words for unbounded and bounded precision arithmetic.

Introduction

This paper proposes a wording for fundamental bit utilities: `std::bit_value`, `std::bit_reference`, `std::bit_pointer` and `std::bit_iterator`. An in-depth discussion of the motivations and an in-depth exploration of the design space can be found in P0237R0. In short, this paper proposes a set of 4 main utility classes to serve as bit abstractions in order to offer a common and standardized interface for libraries and programs that require bit manipulations. These bit utilities both emulate bits that are easy to use for users, and provide an interface to access the underlying words to implement efficient low-level algorithms.

Examples

An implementation of the fundamental bit utilities is available at https://github.com/vreverdy/bit. Before presenting a wording, we illustrate some of the functionalities of the library.

First, `std::bit_value` and `std::bit_reference`:

```
// First, we create an integer
using uint_t = unsigned long long int;
uint_t intval = 42;                                     // 101010

// Then we create aligned bit values and a bit references on this integer
std::bit_value bval0(intval);                           // Creates a bit value from the bit at position 0 of intval
std::bit_reference<uint_t> bref0(intval);               // Creates a bit reference from the bit at position 0 of intval

// And unaligned bit values and a bit references on this integer
std::bit_value bval5(intval, 5);                        // Creates a bit value from the bit at position 5 of intval
std::bit_reference<uint_t> bref5(intval, 5);            // Creates a bit reference from the bit at position 5 of intval
```

```
  // Display them
  std::cout<<bval0<<bref0<<bval5<<bref5<<std::endl;          // Prints 0011

  // Change their values conditionnally
  if (static_cast<bool>(bval5)) {
    bval0.flip();  // Flips the bit without affecting the integer
    bref5.reset(); // Resets the bit to zero and affects the integer
  }
  std::cout<<bval0<<bref0<<bval5<<bref5<<std::endl;          //  Prints 1010

  // Prints the location and the corresponding mask of bit references
  std::cout<<bref0.position()<<" "<<bref0.mask()<<std::endl; // Prints 0 and 1
  std::cout<<bref5.position()<<" "<<bref5.mask()<<std::endl; // Prints 5 and 32
```

Then, with `std::bit_pointer`:

```
  // First, we create an array of integers
  using uint_t = unsigned long long int;
  std::array<uint_t, 2> intarr = {42, 314};

  // Then we create a bit reference and a bit pointer
  std::bit_reference<uint_t> bref5(intarr[0], 5);            // Creates a bit reference from the bit at position 5 of the first element of the array
  std::bit_pointer<uint_t> bptr(intarr.data());              // Creates a bit pointer from the bit at position 0 of the first element of the array

  // We flip the first bit, and sets the second one to 1 with two methods
  bptr->flip();                                              // Flips the bit
  ++bptr;                                                    // Goes to the next bit
  *bptr.set();                                               // Sets the bit

  // Then we advance the bit pointer by more than 64 bits and display its position
  bptr += 71;
  std::cout<<bptr->position()<<std::endl;                    // Prints 7 as the bit is now in the second element of the array

  // And finally we set the bit pointer to the position of the bit reference
  bptr = &bref5;
```

And finally `std::bit_iterator`, which can serve as a basis of bit algorithm:

```
  // First, we create a list of integers
  using uint_t = unsigned short int;
  std::list<uint_t> intlst = {40, 41, 42, 43, 44};

  // Then we create a pair of aligned bit iterators
  auto bfirst = std::make_bit_iterator(std::begin(intlst));
  auto bend = std::make_bit_iterator(std::end(intlst));

  // Then we count the number of bits set to 1
  auto result = std::count(bfirst, bend, std::bit(1));

  // We take a subset of the list
  auto bfirst2 = std::make_bit_iterator(std::begin(intlst), 5);
  auto bend2 = std::make_bit_iterator(std::end(intlst) - 1, 2);

  // And we reverse the subset
  std::reverse(bfirst2, bend2);
```

The count algorithm can be implemented as:

```
// Counts the number of bits equal to the provided bit value
template <class InputIt, class T>
typename bit_iterator<InputIt>::difference_type
count(
  bit_iterator<InputIt> first,
  bit_iterator<InputIt> last,
  const T& value
)
{
  // Initialization
  using underlying_type = typename bit_iterator<InputIt>::underlying_type;
  using difference_type = typename bit_iterator<InputIt>::difference_type;
  constexpr difference_type digits = binary_digits<underlying_type>::value;
  const bit_value input = value;
  difference_type result = 0;
  auto it = first.base();

  // Computation when bits belong to several underlying values
  if (first.base() != last.base()) {
    if (first.position() != 0) {
      result = _popcnt(*first.base() >> first.position());
      ++it;
    }
    for (; it != last.base(); ++it) {
      result += _popcnt(*it);
    }
    if (last.position() != 0) {
      result += _popcnt(*last.base() << (digits - last.position()));
    }
  // Computation when bits belong to the same underlying value
  } else {
    result = _popcnt(_bextr(
      *first.base(),
      static_cast<underlying_type>(first.position()),
```

```
      static_cast<underlying_type>(last.position() - first.position())
    ));
  }

  // Negates when the number of zero bits is requested
  if (!static_cast<bool>(input)) {
    result = (last - first) - result;
  }

  // Finalization
  return result;
}
```

As illustrated in these examples, bit utilities act as a convenient interface between high level code that can use bit manipulation through bit iterators, and low level algorithms that can call dedicated instruction sets and compiler intrinsics.

Proposed Wording

Header `<bit>`

Add the following header to the standard:

```
<bit>
```

Class `std::bit_value`

Add to the `<bit>` synopsis:

```
// Bit value class definition
class bit_value
{public:

  // Types
  using size_type = std::size_t;

  // Lifecycle
  bit_value() noexcept = default;
  template <class T>
  constexpr bit_value(bit_reference<T> val) noexcept;
  template <class UIntType>
  explicit constexpr bit_value(UIntType val) noexcept;
  template <class UIntType>
  constexpr bit_value(UIntType val, size_type pos);

  // Assignment
  template <class T>
  bit_value& operator=(bit_reference<T> val) noexcept;
  template <class UIntType>
  bit_value& operator=(UIntType val) noexcept;

  // Conversion
  explicit constexpr operator bool() const noexcept;

  // Bit manipulation
  void set(bool b) noexcept;
  void set() noexcept;
  void reset() noexcept;
  void flip() noexcept;
};

// Comparison operators
constexpr bool operator==(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator!=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator<=(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>(bit_value lhs, bit_value rhs) noexcept;
constexpr bool operator>=(bit_value lhs, bit_value rhs) noexcept;

// Stream functions
template <class CharT, class Traits>
std::basic_ostream<CharT, Traits>& operator<<(
  std::basic_ostream<CharT, Traits>& os,
  bit_value x
);
template <class CharT, class Traits>
std::basic_istream<CharT, Traits>& operator>>(
  std::basic_istream<CharT, Traits>& is,
  bit_value& x
);

// Make functions
template <class T>
constexpr bit_value make_bit_value(T val) noexcept;
template <class T>
constexpr bit_value make_bit_value(
  T val,
  typename bit_value::size_type pos
);
```

Class template `std::bit_reference`

Add to the `<bit>` synopsis:

```
// Bit reference class definition
template <class UIntType>
class bit_reference
{public:

  // Types
  using underlying_type = UIntType;
  using size_type = std::size_t;

  // Lifecycle
  template <class T>
  constexpr bit_reference(const bit_reference<T>& other) noexcept;
  explicit constexpr bit_reference(underlying_type& ref) noexcept;
  constexpr bit_reference(underlying_type& ref, size_type pos);

  // Assignment
  bit_reference& operator=(const bit_reference& other) noexcept;
  template <class T>
  bit_reference& operator=(const bit_reference<T>& other) noexcept;
  bit_reference& operator=(bit_value val) noexcept;
  bit_reference& operator=(underlying_type val) noexcept;

  // Conversion
  explicit constexpr operator bool() const noexcept;

  // Access
  constexpr bit_pointer<UIntType> operator&() const noexcept;

  // Swap members
  template <class T>
  void swap(bit_reference<T> other);
  void swap(bit_value& other);

  // Bit manipulation
  void set(bool b) noexcept;
  void set() noexcept;
  void reset() noexcept;
  void flip() noexcept;

  // Underlying details
  constexpr underlying_type* address() const noexcept;
  constexpr size_type position() const noexcept;
  constexpr underlying_type mask() const noexcept;
};

// Swap and exchange
template <class T, class U>
void swap(bit_reference<T> lhs, bit_reference<U> rhs) noexcept;
template <class T>
void swap(bit_reference<T> lhs, bit_value& rhs) noexcept;
template <class U>
void swap(bit_value& lhs, bit_reference<U> rhs) noexcept;
template <class T, class U = bit_value>
bit_value exchange(bit_reference<T> x, U&& val);

// Stream functions
template <class CharT, class Traits, class T>
std::basic_ostream<CharT, Traits>& operator<<(
  std::basic_ostream<CharT, Traits>& os,
  bit_reference<T> x
);
template <class CharT, class Traits, class T>
std::basic_istream<CharT, Traits>& operator>>(
  std::basic_istream<CharT, Traits>& is,
  bit_reference<T>& x
);

// Make functions
template <class T>
constexpr bit_reference<T> make_bit_reference(T& ref) noexcept;
template <class T>
constexpr bit_reference<T> make_bit_reference(
  T& ref,
  typename bit_reference<T>::size_type pos
);
```

Class template `std::bit_pointer`

Add to the `<bit>` synopsis:

```
// Bit pointer class definition
template <class UIntType>
class bit_pointer
{public:

  // Types
```

```
  using underlying_type = UIntType;
  using size_type = std::size_t;
  using difference_type = std::intmax_t;

  // Lifecycle
  bit_pointer() noexcept = default;
  template <class T>
  constexpr bit_pointer(const bit_pointer<T>& other) noexcept;
  explicit constexpr bit_pointer(std::nullptr_t) noexcept;
  constexpr bit_pointer(std::nullptr_t, size_type);
  explicit constexpr bit_pointer(underlying_type* ptr) noexcept;
  constexpr bit_pointer(underlying_type* ptr, size_type pos);

  // Assignment
  bit_pointer& operator=(std::nullptr_t) noexcept;
  bit_pointer& operator=(const bit_pointer& other) noexcept;
  template <class T>
  bit_pointer& operator=(const bit_pointer<T>& other) noexcept;
  bit_pointer& operator=(underlying_type* ptr) noexcept;

  // Conversion
  explicit constexpr operator bool() const noexcept;

  // Access
  constexpr bit_reference<UIntType> operator*() const noexcept;
  constexpr bit_reference<UIntType>* operator->() const noexcept;
  constexpr bit_reference<UIntType> operator[](difference_type n) const;

  // Increment and decrement operators
  bit_pointer& operator++();
  bit_pointer& operator--();
  bit_pointer operator++(int);
  bit_pointer operator--(int);
  constexpr bit_pointer operator+(difference_type n) const;
  constexpr bit_pointer operator-(difference_type n) const;
  bit_pointer& operator+=(difference_type n);
  bit_pointer& operator-=(difference_type n);
};

// Non-member arithmetic operators
template <class T>
constexpr bit_pointer<T> operator+(
  typename bit_pointer<T>::difference_type n,
  bit_pointer<T> x
);
template <class T, class U>
constexpr typename std::common_type<
  typename bit_pointer<T>::difference_type,
  typename bit_pointer<U>::difference_type
>::type operator-(
  bit_pointer<T> lhs,
  bit_pointer<U> rhs
);

// Comparison operators
template <class T, class U>
constexpr bool operator==(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator!=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator<(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator<=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator>(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;
template <class T, class U>
constexpr bool operator>=(bit_pointer<T> lhs, bit_pointer<U> rhs) noexcept;

// Make functions
template <class T>
constexpr bit_pointer<T> make_bit_pointer(T* ptr) noexcept;
template <class T>
constexpr bit_pointer<T> make_bit_pointer(
  T* ptr,
  typename bit_pointer<T>::size_type pos
);
```

Class template `std::bit_iterator`

Add to the `<bit>` synopsis:

```
// Bit iterator class definition
template <class Iterator>
class bit_iterator
{public:

  // Types
  using iterator_type = Iterator;
  using underlying_type = typename _cv_iterator_traits<Iterator>::value_type;
  using iterator_category = typename std::iterator_traits<Iterator>::iterator_category;
  using value_type = bit_value;
```

```
  using difference_type = std::intmax_t;
  using pointer = bit_pointer<underlying_type>;
  using reference = bit_reference<underlying_type>;
  using size_type = std::size_t;

  // Lifecycle
  constexpr bit_iterator();
  template <class T>
  constexpr bit_iterator(const bit_iterator<T>& other);
  explicit constexpr bit_iterator(iterator_type i);
  constexpr bit_iterator(iterator_type i, size_type pos);

  // Assignment
  template <class T>
  bit_iterator& operator=(const bit_iterator<T>& other);
  bit_iterator& operator=(iterator_type i);

  // Access
  constexpr reference operator*() const noexcept;
  constexpr pointer operator->() const noexcept;
  constexpr reference operator[](difference_type n) const;

  // Increment and decrement operators
  bit_iterator& operator++();
  bit_iterator& operator--();
  bit_iterator operator++(int);
  bit_iterator operator--(int);
  constexpr bit_iterator operator+(difference_type n) const;
  constexpr bit_iterator operator-(difference_type n) const;
  bit_iterator& operator+=(difference_type n);
  bit_iterator& operator-=(difference_type n);

  // Underlying details
  constexpr iterator_type base() const;
  constexpr size_type position() const noexcept;
  constexpr underlying_type mask() const noexcept;
};

// Non-member arithmetic operators
template <class T>
constexpr bit_iterator<T> operator+(
  typename bit_iterator<T>::difference_type n,
  const bit_iterator<T>& i
);
template <class T, class U>
constexpr typename std::common_type<
  typename bit_iterator<T>::difference_type,
  typename bit_iterator<U>::difference_type
>::type operator-(
  const bit_iterator<T>& lhs,
  const bit_iterator<U>& rhs
);

// Comparison operators
template <class T, class U>
constexpr bool operator==(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator!=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator<(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator<=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator>(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);
template <class T, class U>
constexpr bool operator>=(const bit_iterator<T>& lhs, const bit_iterator<U>& rhs);

// Make functions
template <class T>
constexpr bit_iterator<T> make_bit_iterator(T i);
template <class T>
constexpr bit_iterator<T> make_bit_iterator(
  T i,
  typename bit_iterator<T>::size_type pos
);
```

Helper struct `std::binary_digits`

Add to the `<bit>` synopsis:

```
// Binary digits structure definition
template <class UIntType>
struct binary_digits
: std::integral_constant<std::size_t, std::numeric_limits<UIntType>::digits>
{};
```

Acknowledgements

The authors would like to thank Howard Hinnant, Jens Maurer, Tony Van Eerd, Klemens Morgenstern, Vicente Botet Escriba, Tomasz Kaminski, Odin

Holmes and the other contributors of the ISO C++ Standard - Discussion and of the ISO C++ Standard - Future Proposals groups for their initial reviews and comments. Vincent Reverdy and Robert J. Brunner have been supported by the National Science Foundation Grant AST-1313415. Robert J. Brunner has been supported in part by the Center for Advanced Studies at the University of Illinois.